and work to manipulate networks of probabilistic data by means of rules, inferences and reasoning systems. These are similar to an extension of probabilistic logic networks to a generalised system where data is automatically managed.

Atomese, the programming language. The above programming-like language is informally referred to as Atomese. Developers describe Atomese as a strange mash-up of SQL, prolog/datalog, lisp/scheme, haskell/caml and rule engines. Most engines provide basic atoms that represent relations like similarity, inheritance; logic like Boolean or, there-exists; for Bayesian and other probabilistic relations; intuitionist logic like absence and choice; thread execution; and expressions including variables, mapping or constraints. As a programming language, Atomese is probably inefficient, slow and not scalable, but powerful as an input to automation sub-systems.

Putting the better of the two together. AtomSpace and Atomese together let you integrate your needs for dependent systems like machine learning, natural language processing, motion control and animation, planning and constraint solving, pattern and data mining, question answering and common-sense systems, and emotional and behavioural psychological systems. You can start off with the examples section in OpenCog to get a hand over Python and Scheme bindings, pattern matcher, rule engine and other different atom types, and learn how to use these for solving various tasks.

AtomSpace, a boon in disguise

Simplifying things a little further, you have to work with these AtomSpaces instead of individual atoms, as there are advantages of de-duplication, fast searches by atom type or name, automated attention-allocation management and automated atom fetch and save to a persistent database, and automatic sharing of this database by multiple machines in a cluster.

You can put together multiple AtomSpaces as different threads on the same machine, to perform complex tasks. Atoms can be uniquely identified by handles, and AtomSpace has signals that are delivered when an atom is added or removed, which can then be configured to trigger other actions. A persistence backend allows multiple AtomSpaces on multiple networked servers to store atoms on a disk or different machines, and communicate and share atoms.

Atomese, the programming language

Atomese is the provisional name given to the concept of writing programs with atoms, that is, defining functions and structures using FunctionLink, ExecutionOutputLink, PutLink, GetLink and so on. It vaguely resembles Prolog but with types and kind-of functional-programming.

One motivation for Atomese is the need to implement procedural scripting and behaviour trees within OpenCog.

Some exploratory work is being done on making more elegant or powerful frameworks for coding Atomese, for example, syntactically sugaring Scheme or Python shells for making Atomese more concise and transparent when coded in those languages, or doing something more radical like making AgdaAtomese exploiting dependent-type theory.

—Courtesy: http://wiki.opencog.org/w/Atomese

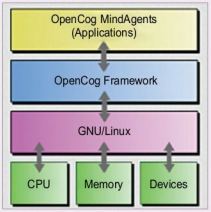

Fig. 3: OpenCog framework

Managing generated data

While dealing with any kind of system, memory management becomes very important. How do you handle the vast expanse of data that is generated? What do you do when there is a lot of old data already? How do you prioritise? OpenCog terms this entire process as attention allocation. Pieces of knowledge are weighed relative to one another, taking into account how important a certain piece of data has been in the past and how important it is currently. This calculated analysis then guides the processes of:

Storing knowledge. What to store in memory, what to store locally on a disk and what can be off-loaded to other machines

Deleting information. To guide the process of forgetting data that is no longer needed or what has been integrated into the system in other ways

Reasoning. To prioritise which threads of data should be taken into account while making any kind of decision, to analyse the combinatorial explosion of potential inference paths and cut off the ones with very low importance.

OpenCog uses available hardware resources to run software, taking a cue from neuroscience, cognitive psychology and computer science. One principle that OpenCog works on is that, intelligence is not owned by a single algorithm. Instead, it is a culmination of a large number of algorithms that work in cognitive synergy. The intelligent agent's motivations, drives, emotions and decisions are driven, higher-level features of cognition emerge based on the way the system is setup and its interaction with the environment. EFY

Priya Ravindran is M.Sc (electronics) from VIT University, Vellore, Tamil Nadu. She loves to explore new avenues and is passionate about writing